

Report of the Project

In this file we will describe all the steps we have followed to obtain the solution starting from the baseline.

Dataset

Starting from the baseline architecture, we opted to train and validate the model using the data from just the S1 folder, which contains the data obtained from the movements of person 1 always in the same room divided in 3 days, we used the first 2 to train and the last one to validate / test.

- in this way we just train using always the same room / person so the generalization to other rooms / person can be done by exploiting other of them as the test set

The data for a given folder are divided other than for the day (which is represented by the letter a, b or c) also by:

- The type of movement registered, which can be either jump, walk, stay still, sit, run, ...
- the channel that has been used for that track, indeed we have 4 channels for each registration, in this way we are less dependent by the position of the router in the room

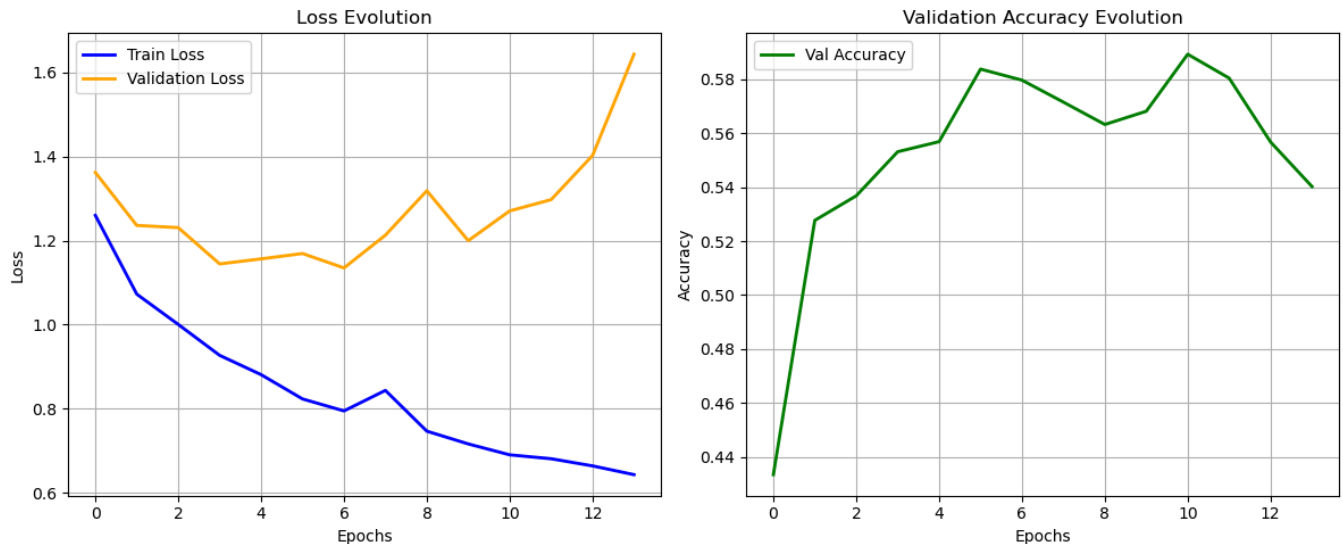
The data registered for each folder, day, movement and channel are composed by a couple of minutes of registration of the same activity and what we are interested in are windows of a couple of seconds, indeed in input we want matrices of 340 x 100.

So what we did was take each matrix composed by a couple of minutes of data and split them into smaller matrices of 340x100 so by dividing on the first dimension we obtained the required format But just by partition each file we would have got only a few dozens of data.

A better idea consists in using sliding windows that every 5 columns take another screenshot of the matrix, in this way we obtain a lot of data that are correlated with each other but have the important information in different position, making the data less prone to overfit since it has to detect the element indendently from where it is in the matrix and at the same time give us more data to train and validate the algorithm. We opted for a stride equal to 5 and not smaller since we found it a good tradeoff between having enough data to train and validate the model on and having some variety in the data, causer otherwise with a pooling layer the invariance would make following screenshots basically the same so it would make the network overfit on data it has already saw which is a no-sense.

Baseline Model

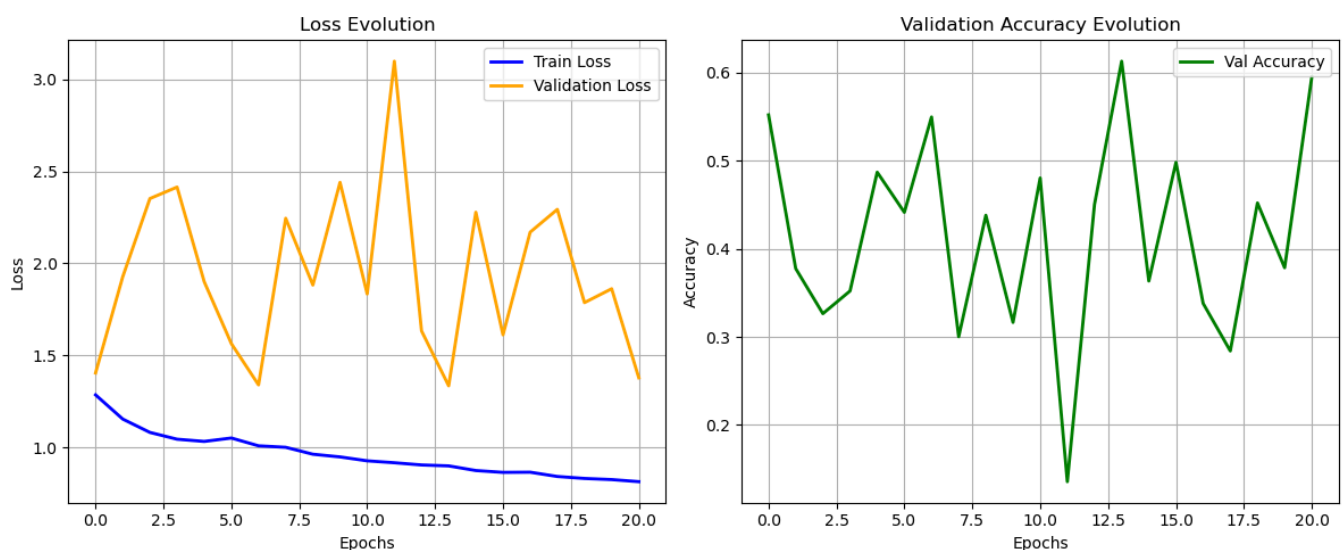
after implementing the baseline model, we have obtained the following results:



We now want to make the architecture more robust by adding some batch normalization between the layers (regularizing and reducing covariate shift maybe) Using some type of pooling before the flattening of the maps, the idea is to change the convolution layer after the concatenation such that it has more filters so the flattening is more "clean" while reducing height / width.

- The Pooling layer we opted to use is the adaptive max pool since it is better than the average in the case of the spectrogram since it is more able to identify the peaks while the average risks to dilute the changes making them noisier, moreover the maxpooling offer a great invariance to translations making it suitable for our case where we slide the window between the different instances.

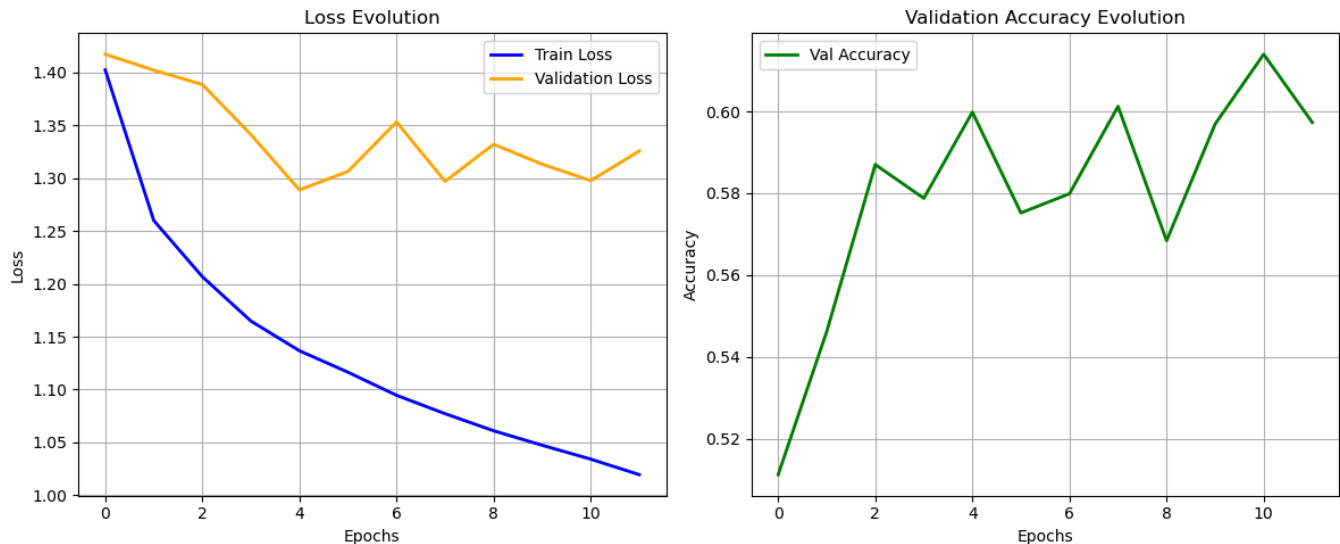
Moreover we change the loss function to make it less sure about its own decision such that it is more capable of generalizing and less prone to overfit We also put some kind of L2 regularization by increasing the weight decay cause u never know maybe its a good idea The last optimization in the training process was to make the learning rate adaptive in a temporal way such that it is faster at the beginning and then slower when it need to find the optimum. To do so we opted to use the Cosine Annealing. We obtained the following results:



Which we can see tends to overfit in a reduced way but there is still a lot of noise in it, which we would like to remove

To do so we try to do two different things, change the pooling to not be adaptive but such that it just have a kernel size equal to 2 and stride = 2 so it reduce by 4 the dimension. Moreover we opted to use the

InstanceNormalization instead of the BatchNormalization which basically normalize based on the value of the cells of the matrix instead of normalizing based on the value of the cells in different instances in the batch, which theoretically should make the model focus more on the content since each value in the matrix is normalized by the values in that matrix making matrices with different type of noises behave in the same way even if the user is positioned in different ways from the transmitter/receiver of signals. We also opted in implementing gradient clipping since we saw that in case of strange batches of data we obtain way worst performances and make the model way worst with the risk of not coming back (collapse of the model). With this new updates indeed we obtained a way better model that is less affected by noise and way more stable:



The last thing we tried to implement on the standard convolutional architecture was to increase the number of training data by exploiting data augmentation, indeed in the case of the spectrograms we have specific techniques like the masking block

- Time Masking -> set to zero values in a given row, the model has to understand the activity even in case of temporal holes in the signal
- Frequency Masking -> set to zero values in a given column, the model has to understand the activity even in case of a specific frequency that is affected by a total interference

We have also implemented the confusion matrix to have more information about what is happening in our training and which are the classes that we are better at identify and which are the ones we have more difficulties:  Confusion Matrices 2nd Version What we can notice is that some classes seems more difficult to predict than others:

- the network confuse L for E, meanwhile is very sure when it comes to predict E , seems like prediliges E between the two when it is undecided, probably given by the fact that the samples with E activities are more numerous than the ones with E, but still E has a decent amount of samples. If E represents the absence of movements, then L should be an activity with a very low energy and the network looking at the low energy in the doppler spectrogramm opts the safest way where there are more samples
- Jump Run and Walk are quite similiar to each other given the fact that they are big movements, especially jump have a low accuracy since it is often confused for walking
- H and C are quite interchangeable since they are both the only other class that the model confuse the other with, so they could have a very similiar spectral firm
- S is confused for almost every class that requires a quite low amount of energy (so all other than jump, walk and run)

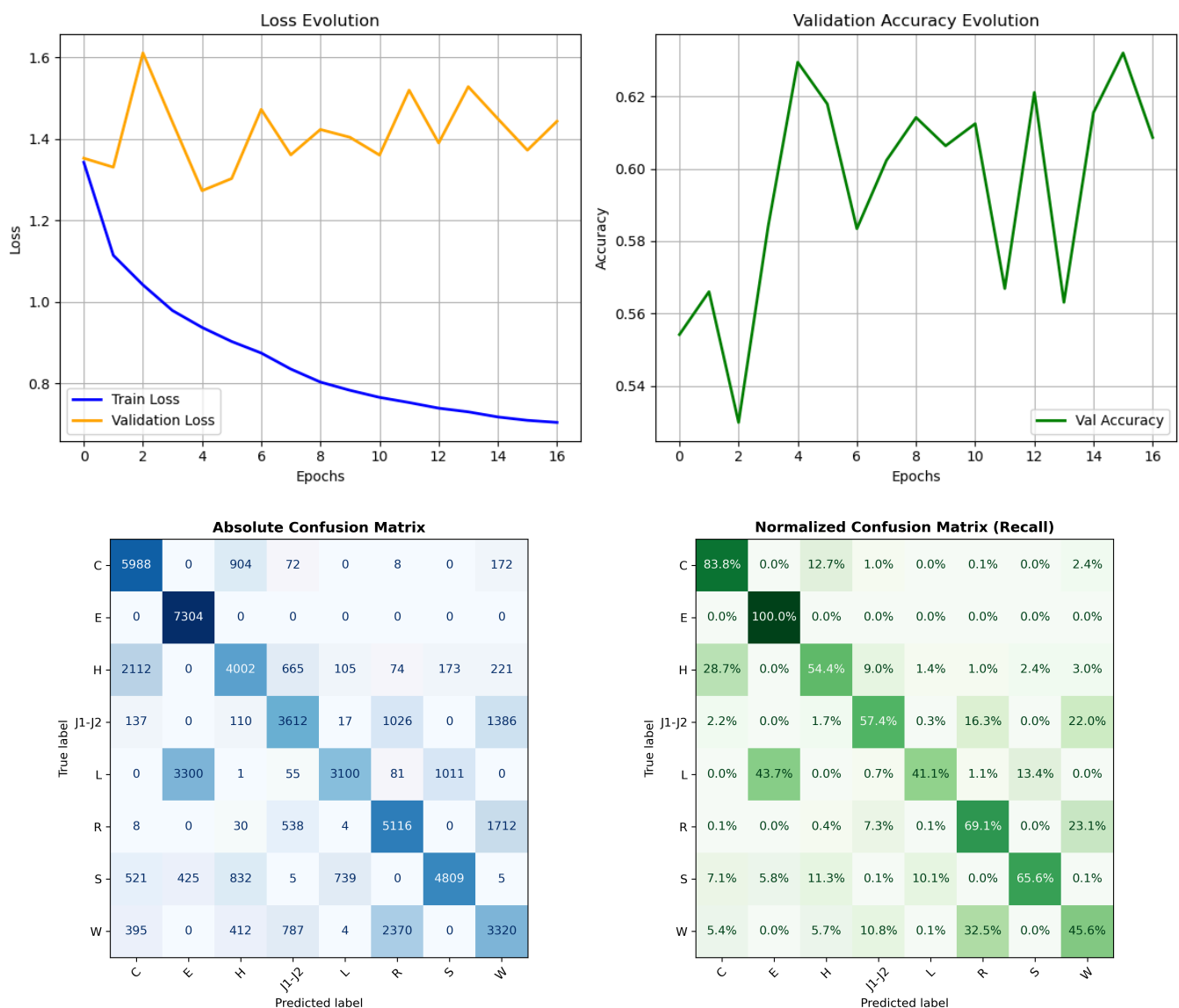
Our hypothesis is that the problem resides in the fact that the convolutional network is not able to classify correctly activities of similar level of energy since they miss part of the temporal information and by the fact that movements at low energy levels could be confused by some background noise

So what we want to do now is to try to implement some kind of recurrency in the network such that we can extrapolate more information from each matrix.

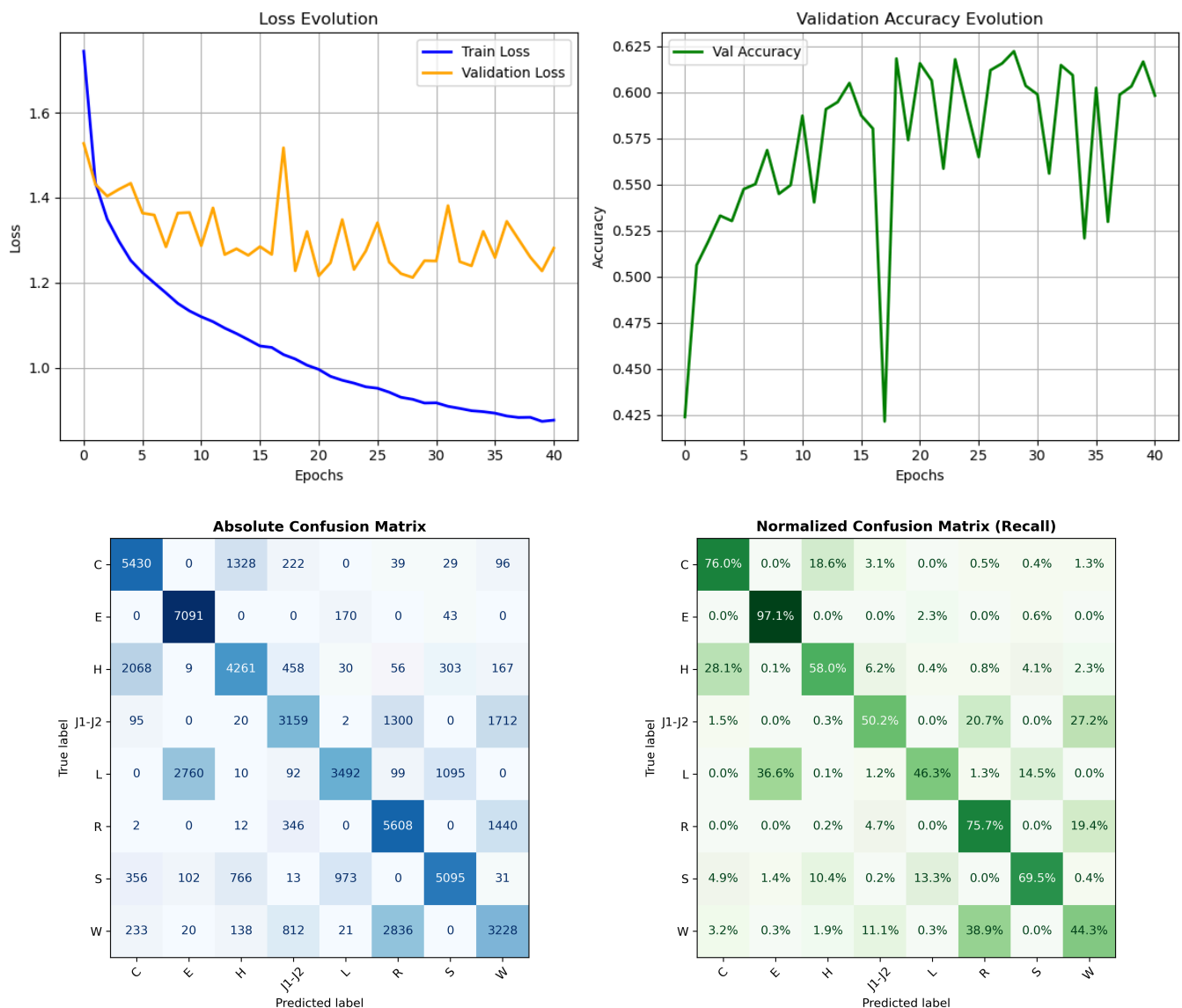
The idea is to put some kind of recurrent network after the inception layer, in this way we make the convolution filters extrapolate the useful patterns and then we pass those patterns in input to the recurrent layer that learn the evolution of the action in time such that we have more information to base our prediction on.

Initially we implemented a bidirectional LSTM layer where we basically take in input the output of the convolutional network and indicate the temporal dimension as the one we want to use to extract the recurrence and unify the filter and frequencies dimensions as the other dimension that we want to extrapolate informations on. Indeed instead of flattening all dimensions together, we preserve the time dimension as a sequence.

The bidirectional processing let the model leverage all the temporal context in both direction which is useful since we have the hole spectrogram to compute at the beginning and give us more informations. The output of the LSTM is averaged across the time dimension to obtain a single global representation to pass to the head projection used to merge the informations of the 2 directions together:



Looking at this model we can have mixed feelings, indeed we can notice that we obtain a significative improvement in the trainig indeed the model was increasingly improving in the training loss, the problem however resides in the validation loss, which suggests us that we are quite overfitting, so the next step is to try and reduce this problem as much as possible. The first step we opted to adopt is to increase the stride of the sliding window in the training set, indeed using a recurrent network, having too similar matrices can only make the model overfit on those that he sees more frequently. So we will use stride = 25 on the training while keeping the one of 5 for the validation / test since we want to have results that are less affected by randomness as possible The second technique we decided to adopt is to make the output of the LSTM not only focus on the mean of the different instances of time, but also the max and variance, in this way we have a better understanding of the distribution which could help us to distinguish the cases like the ones between lie down and empty we also tried to add minor regularization like increasing weight decay and dropout in the head projection. This are the results:



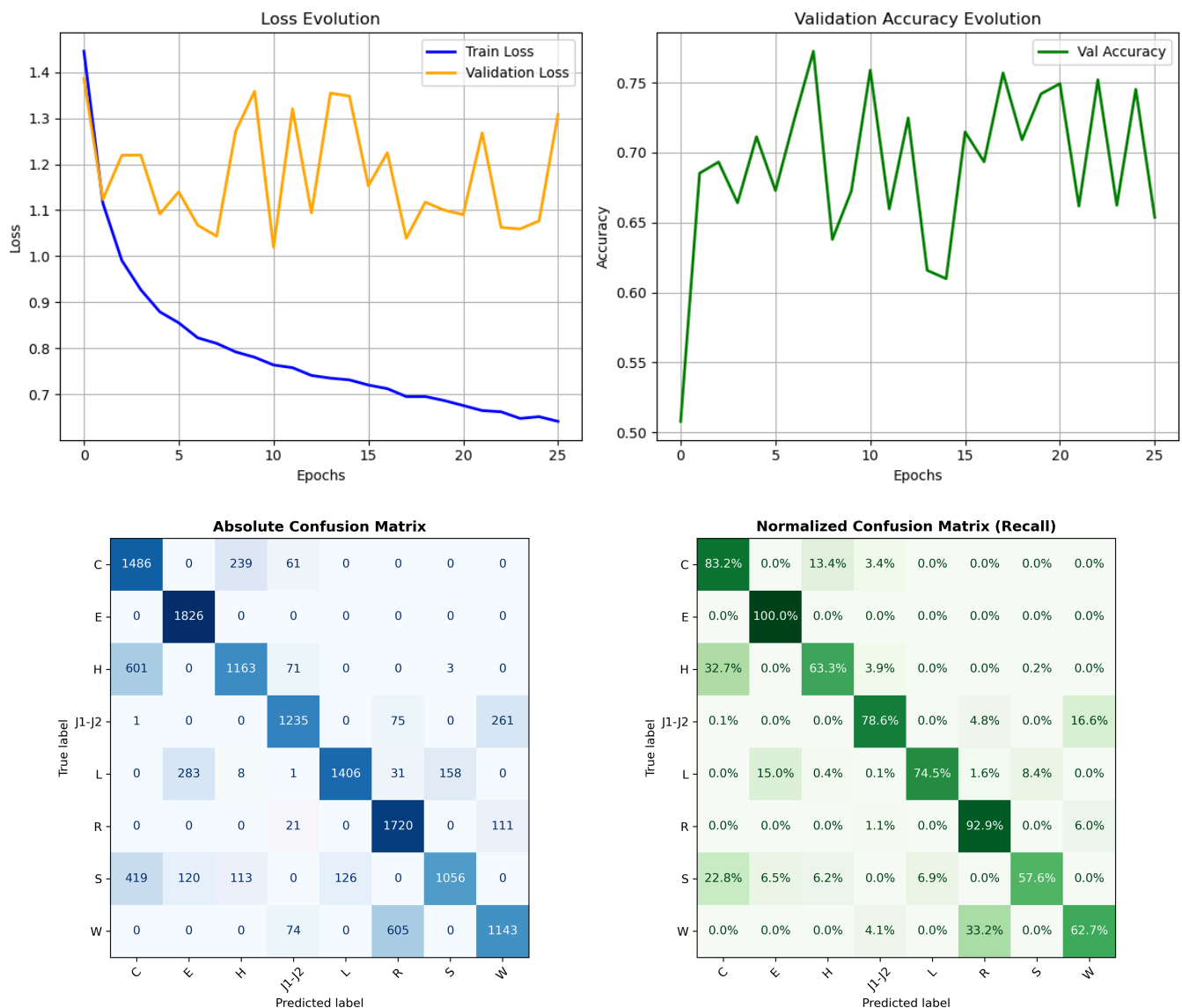
Which we can say that are great since we reduce the overfit given by the fact that the validation loss decrease for 50 epoches instead than after 5 epoches it already start overfit too much which is a good result, moreover the model seems more stable and we can notice an improvement of the performances in those class we where more concerned of.

But still the results are quite poor so we opted to change the approach on how we read the data for the training and make the predictions. Indeed we have 4 different channels for each room/activity/day monitored, and until now we just took those 4 different instances as independent instances to train the model on, the

problem on this are 2: no assumption of iid (which is also already affected by the sliding windows but in this case those are exactly the same movement monitored at the same time), and we don't leverage the fact that we have multiple information on a given instance, which is quite a pity. The idea is to now use the channel fusion and see if it actually fixes our problems of accuracy. So what we basically do is to take a kind of majority vote between the different channels on the given action on the same given moment and take that as predicted label.

To do so we have lowered the dimension of the batch otherwise the epochs would have a very low cardinality and implemented a new dataset that put in a list all the matrices that register the same thing but with different channels such that we have all of them in the same batch all the times, we have also increased the number of epochs of train since we have less instances in each epoch (indeed if before we have 4 times the instances, now we group those as one instance to make the prediction): LATE FUSION.

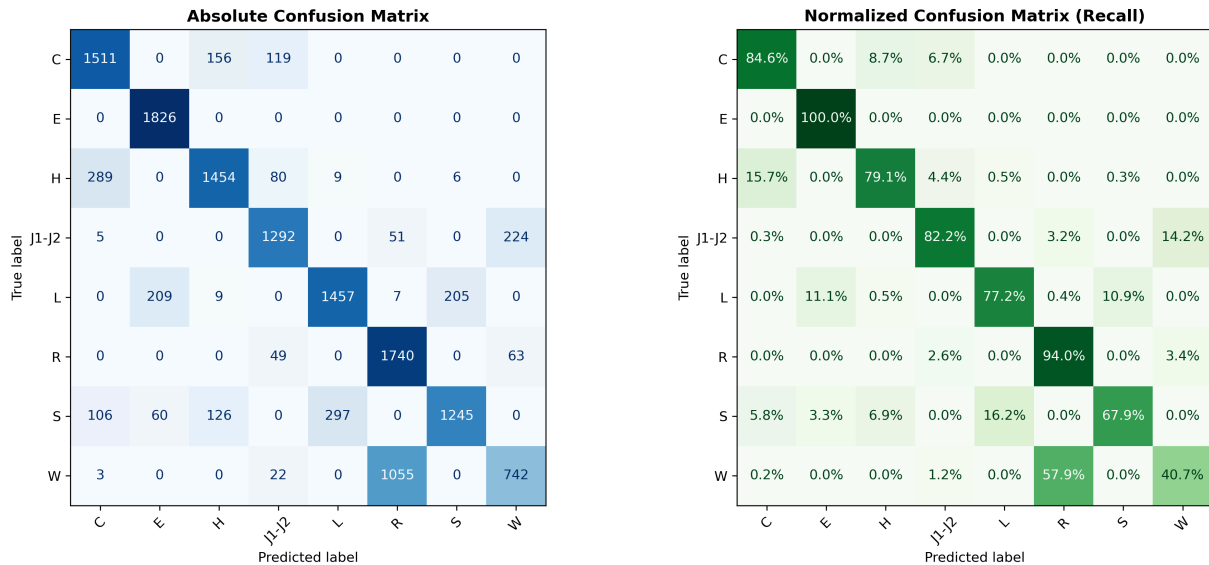
At first we thought that to optimize the training and validation, computing the loss directly on the mean of the logits was a better solution to obtain convergence in the model, however the empirical comparison shows that the aggregated optimization brings a rapid overfit on the training campaign making the model reach a max accuracy of 75% with very high variability.



We can notice that the biggest problem of this model is the fact that it overfits a lot on the data, we would like to get a model that is more capable of generalizing data.

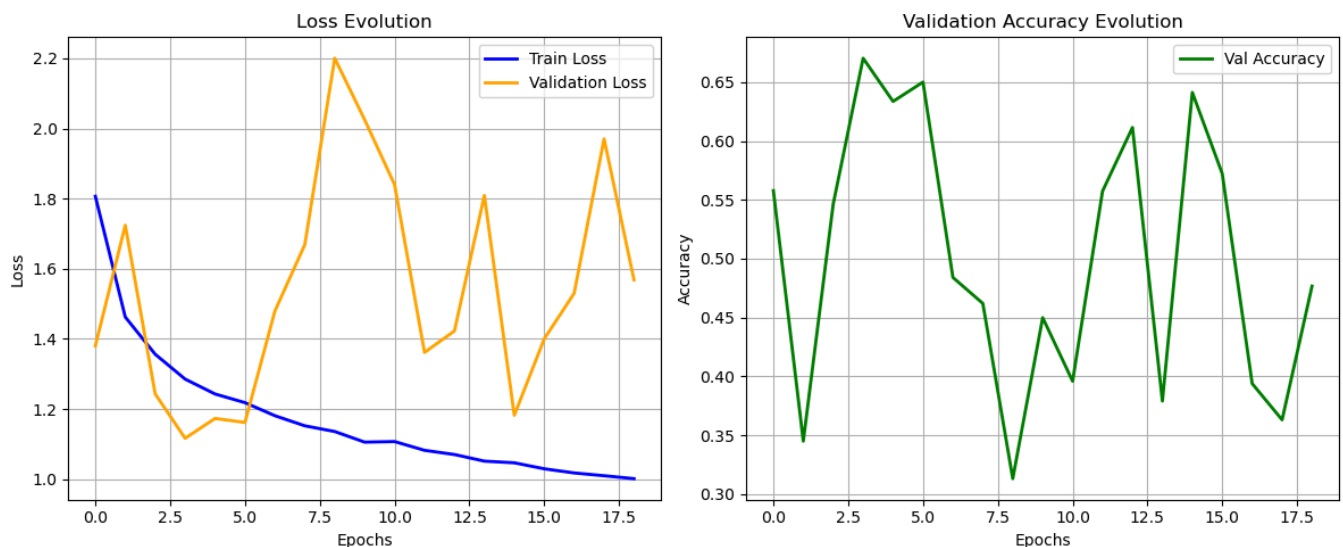
Instead making the training on the different channels and then take the aggregation of them in the validation works pretty well as regularization parameter since it makes the network extract the significant features in an independent fashion between channels decreasing the co-adaptation that makes the network generalize way better in the domain shift with an accuracy of 85% and more stability.

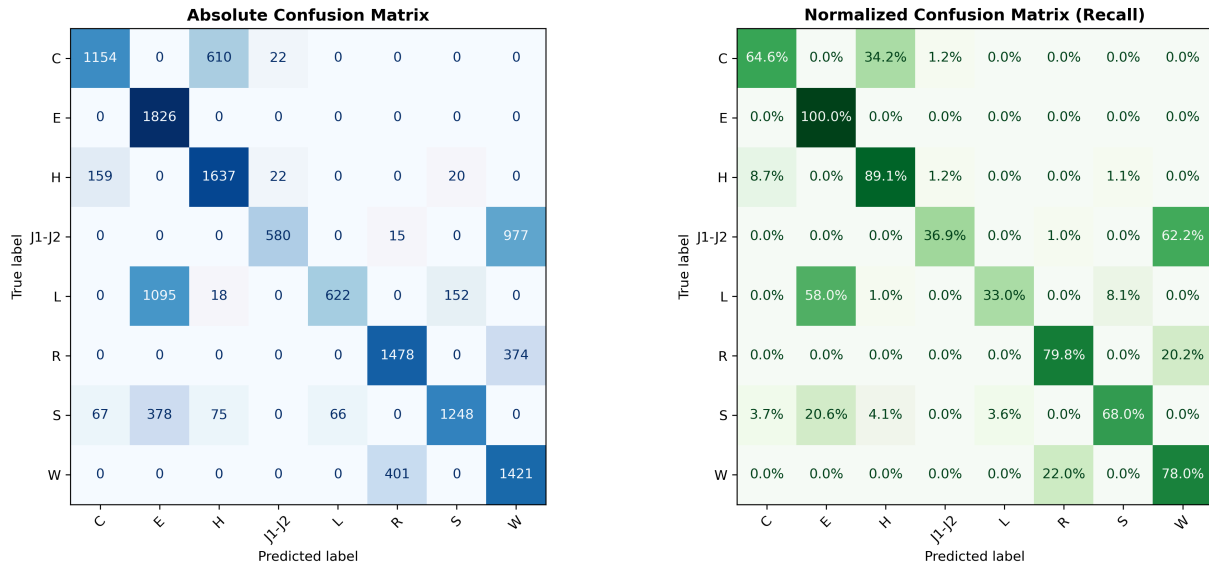
The following are the results of the obtained model:  Recurrent Curves Version2



We can notice that with this correction the model performs way better than before, the problems given by the lie down activity and the jump activity are way less and we have a more precise model that doesn't overfit at all since the validation loss is a random walk around the training loss, maybe one thing we could improve is to reduce the variability, but since now we have moved to a lower number of batches due to the fact that we need 4 instances to classify a given moment it is quite normal.

Another thing we wanted to try is to now remove the instance normalization since it kind of make the filters treat in the same way matrices with different energies since we normalize the matrix and maybe now that we have implemented recurrency if elements are more distinct it could actually benefit the model To solve this without regressing on training stability, we transitioned from Instance Normalization to Global Z-Score / BatchNorm, preserving cross-sample energy relationships. The following are the results of this refinement:





We can notice that this attempted improvement works way worst than the previous architecture indeed now the model is not able to classify jump and lie down, moreover now it exchange C as H. This could be due to the initial intuition that batch norm tends to offuscate the data in a way that is bad for spectrograms since it just create noise in the background given by other batches so we will stay with our instance normalization.

To investigate the role of signal energy in activity classification, we conducted an ablation experiment comparing two normalization strategies on the multi-channel Late Fusion architecture:

- Instance Normalization (per-window contrast scaling)
- Global Z-Score Standardization (computed on the training set) combined with Batch Normalization

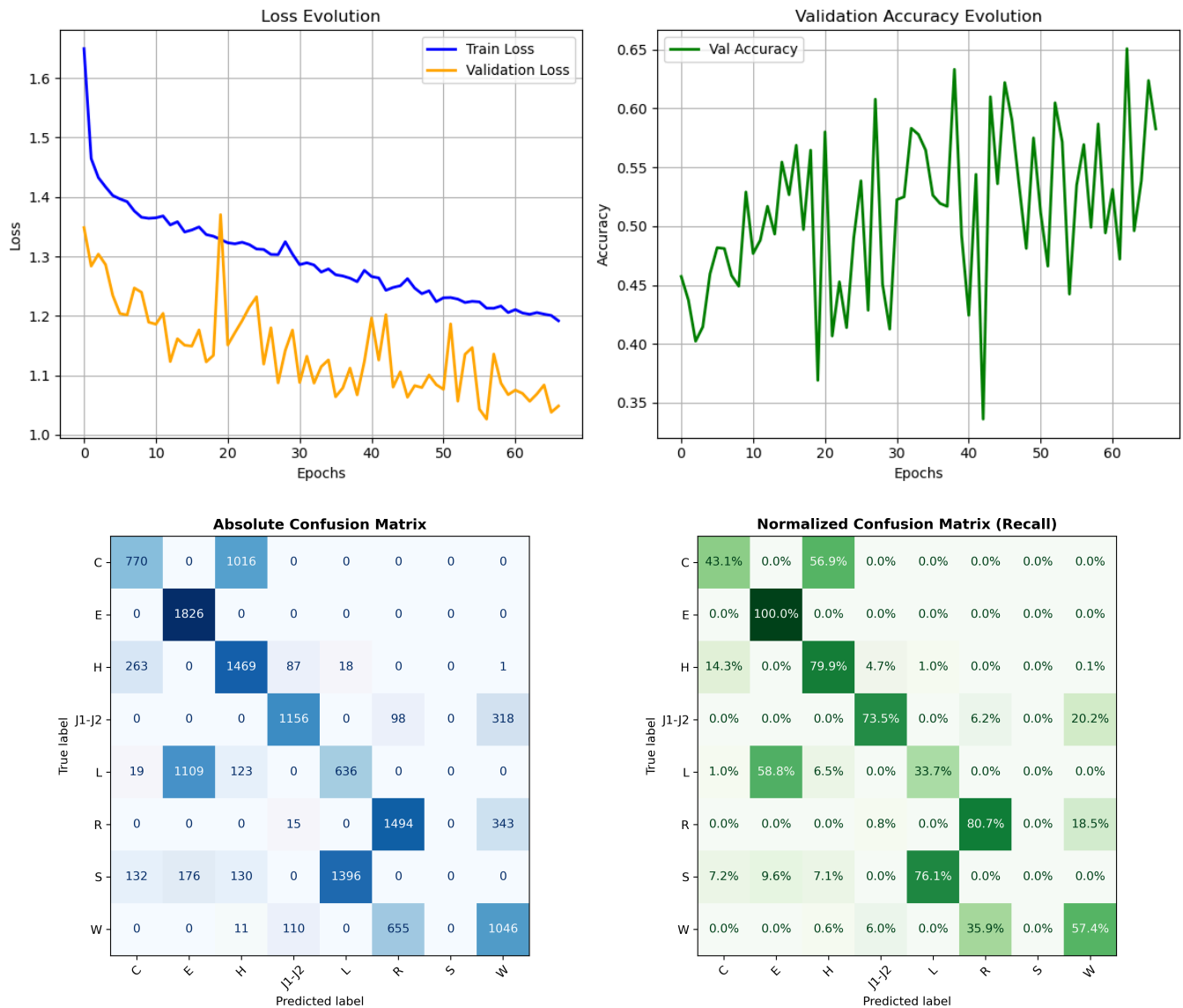
The experimental comparison yielded a clear result:

- Instance Normalization achieved superior performance, reaching ~80% validation accuracy with stable validation loss curves. It effectively isolated micro-Doppler frequency shifts, enabling the model to correctly identify *Jump* (71.3%) and *Lie down* (80.6%).
- Global Z-Score + BatchNorm caused a severe model collapse. As shown in the validation curves, the validation loss exploded past 2.5, with accuracy dropping to 33%. Crucially, the confusion matrix reveals that without per-sample gain normalization, 82.7% of *Jump* instances were misclassified as *Walk*, and 71.3% of *Lie down* instances collapsed into the *Empty* class.

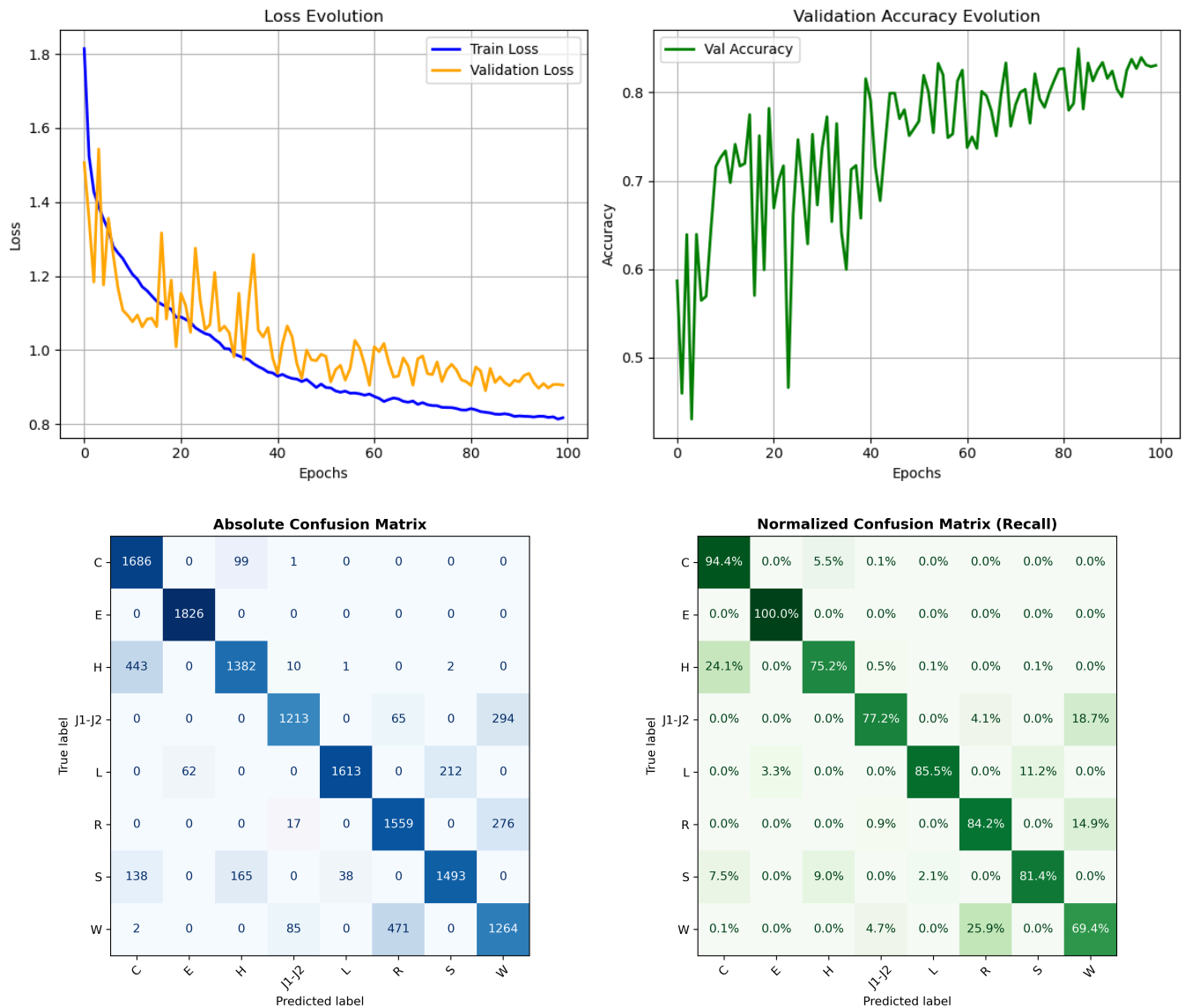
In Wi-Fi CFR/Doppler spectrograms, absolute signal amplitude is heavily corrupted by path loss, distance to antennas, and hardware Automatic Gain Control (AGC). A global Z-score leaves these position-dependent gain variations intact, forcing the model to classify based on signal power rather than movement patterns.

Conversely, Instance Normalization acts as a local contrast enhancement, making the representation invariant to subject distance while preserving the structural micro-Doppler velocity signature. Consequently, Late Fusion with Instance Normalization was selected as our final optimal architecture.

To check if all the updates we made on the architecture were actually helpful or its just the channel fusion to make the magic, to do so we implemented it in the baseline architecture using the training parameter we used in the current model and obtain the following results:

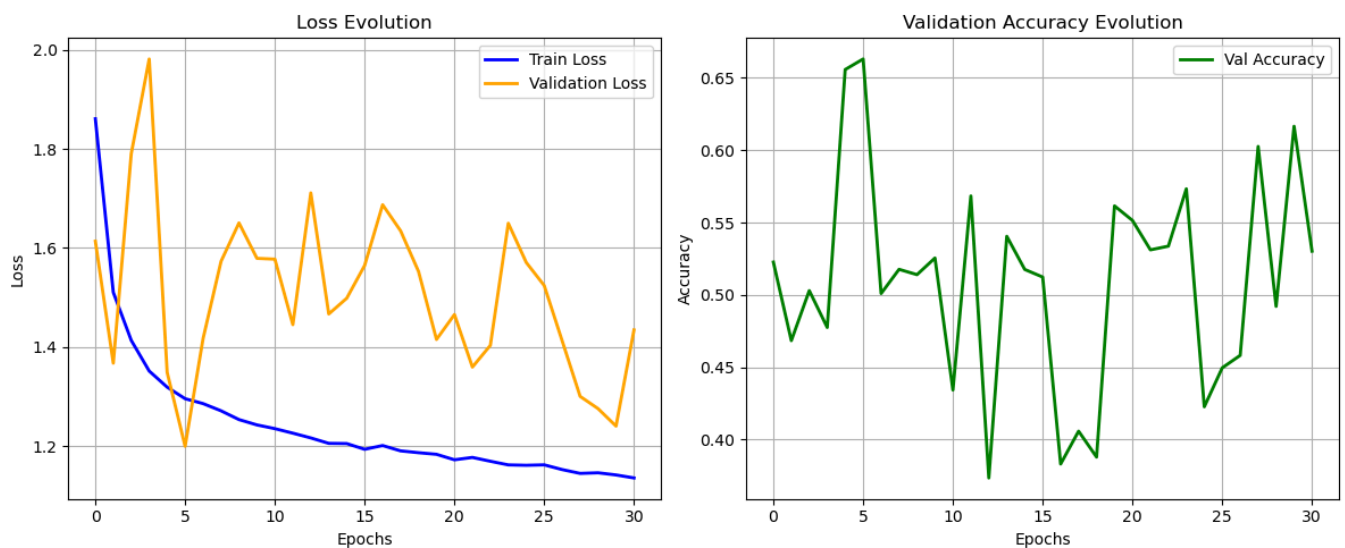


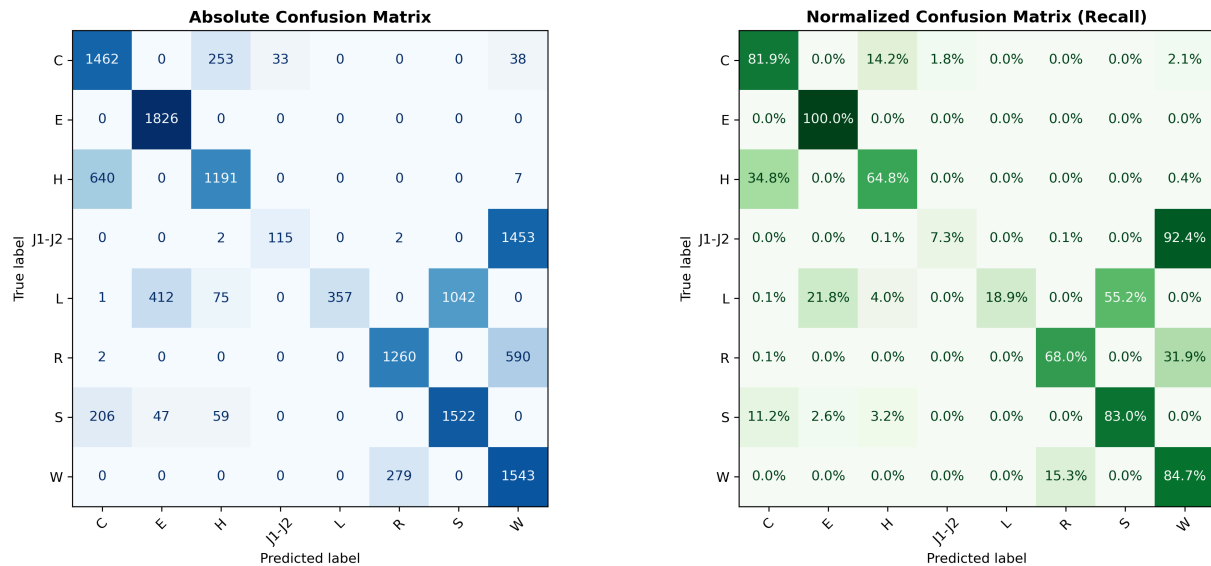
We would now to implement a new technique on how to deal with the results of LSTM and aggregate the temporal informations: Instead of relying on global statistics, we implement a temporal attention mechanism to identify the features we are more interested in for a given case and which are less interesting, which should make us increase the performance theoretically. especially in the case of the jump / walk and the lay / empty activities, that are the one our current model makes more difficulties to learn. To do so we designed a temporal attention layer that computes the relevance score for each temporal frame through a ff layer that is then passed to a softmax function to normalize the values. The final vector is constructed as the weighted linear combination of the frame-level lstm representations We opted to keep the standard deviation with the attention to get additional information on the type of movement (continued or instantaneous). While the BiLSTM captures the sequential dependency and temporal context across adjacent frames, the Temporal Attention layer acts as an adaptive pooling mechanism. It replaces static global operations (like time-averaging) by dynamically selecting and emphasizing the most informative LSTM hidden states while suppressing stationary background frames. The results are the following:



Accuracy : 83.48% Macro F1-Score : 83.52%

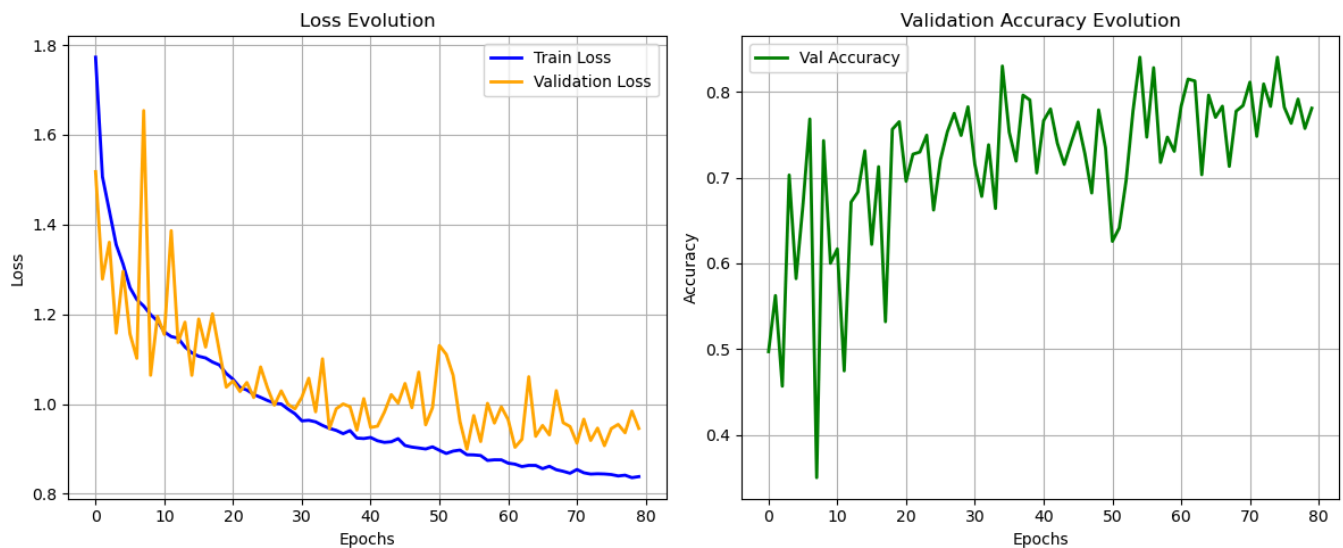
In alternative to this we tried to substitute the LSTM with the attention with a multihead transformer encoder, giving us very bad results, probably due to the fact that the transformer is better in obtaining the context in case of longer sequence and is more prone to overfit in case of less samples.

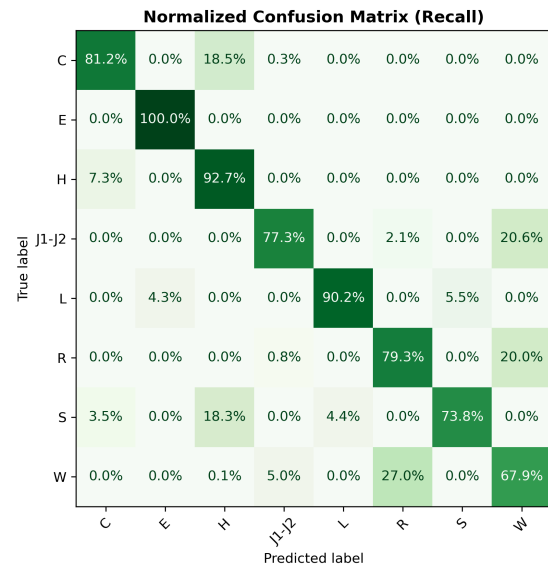
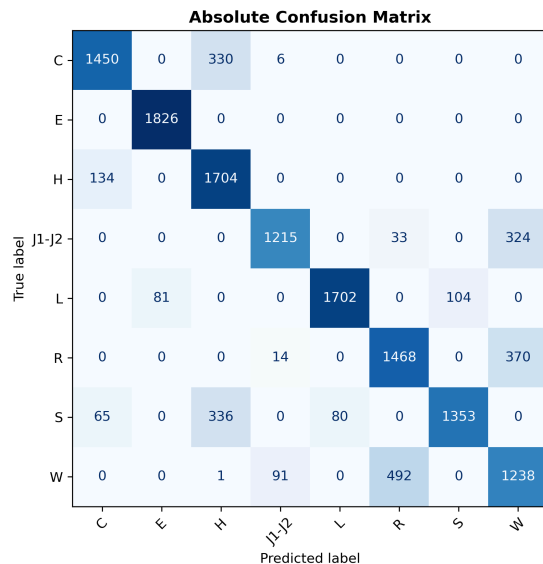




Indeed we can notice that it is very sure about some classes, making it too confident and making wrong decisions

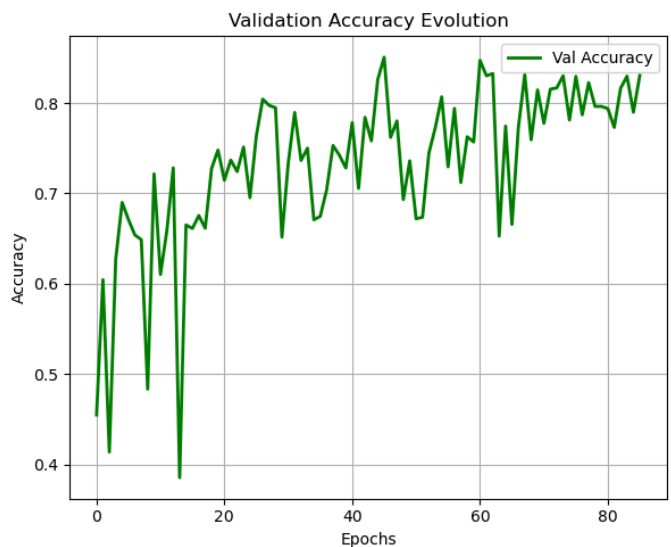
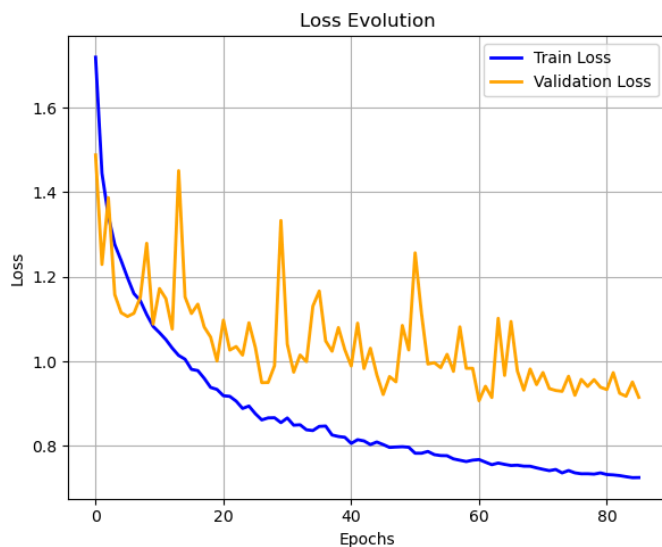
Which we can notice improve a lot the model making it perform way better than before. The biggest problem we can notice on this model is that it still make difficulties in classify the high energy movements correctly, indeed it seems to confuse them quite oftenly. One first improvement we implemented for this purpose is to try choosing the final label in a better way, indeed there could be a channel that is more confident about its decision than the other 3, so we should rely more on him than on the others, to do so we implemented a softmax entropy weighting.

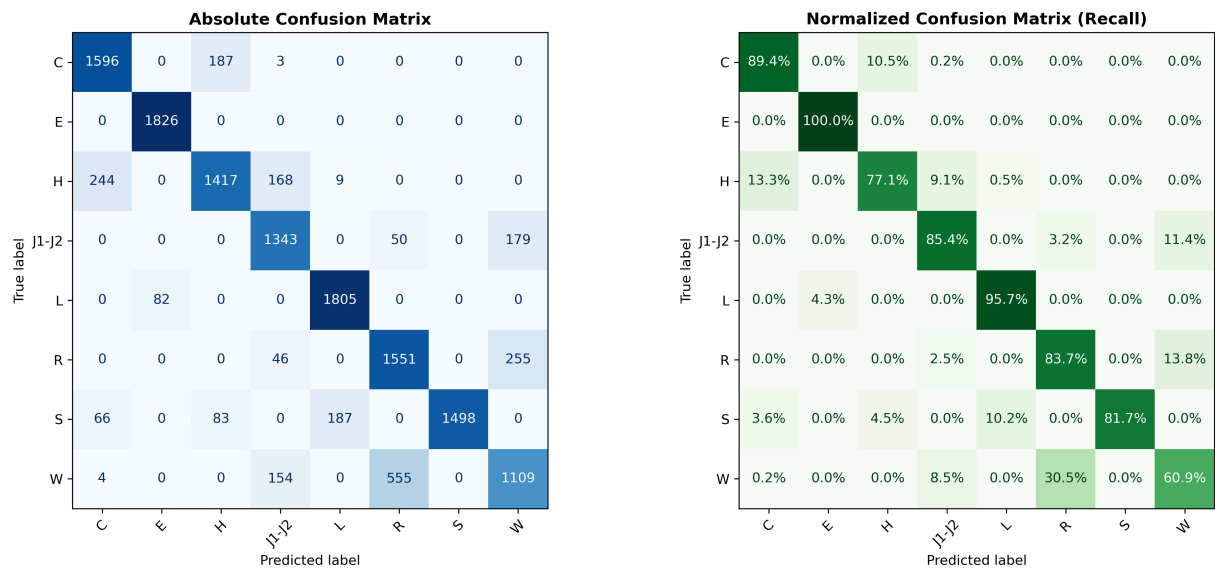




we can notice that we have less difficulties in distinguish H from C but now we have more difficulties in distinguish C from H and the other results are kind of the same but a little worst, so we can say that just taking the mean of the 4 antennas instead of the weighted average is better since it is less penalized in case the antenna that is sure has made a wrong decision

a second improvement we implemented is specific to try improve the classification in case of jump/walk/run, which is to also obtain from the input the derivative from subsequent frame, in this way we can catch the acceleration of changements in a fixed way.





Accuracy : 84.24% Macro F1-Score : 84.05% which has some improvement in the classification of J but is a little worst in classifying S